

Apéndice C

Especificación de diseño

C.1. Introducción

Este apéndice recoge la especificación de diseño de IUCE Reservas y describe cómo está construido el sistema en tres planos complementarios: el diseño de datos (entidades, relaciones e índices), el diseño procedimental (flujos y máquina de estados de la reserva) y el diseño arquitectónico (capas, rutas, seguridad y despliegue). Mientras que el Anexo B establece qué debe hacer el sistema y por qué, este anexo documenta cómo se ha resuelto, tal y como está implementado.

C.1.1. Visión general del stack

Tabla 21. *Visión general del stack.*

Capa	Tecnología
Frontend	Next.js 14 (App Router) + TypeScript 5 · React 18 (Server Components por defecto)
Estilos / UI	Tailwind CSS 3 · componentes propios con class-variance-authority (sin librería externa de UI)
Backend / API	Next.js API Routes (REST) + Prisma 6 (ORM)
Base de datos	PostgreSQL 16
Autenticación	NextAuth.js v5 · magic link passwordless @usal.es
Email	Resend (plantillas HTML propias)
Despliegue	Producción: Docker (app + PostgreSQL) tras Apache 2 (proxy inverso) + Let's Encrypt en el CPD-USAL · Desarrollo: Docker (PostgreSQL)

Todo el sistema se escribe en un único lenguaje (TypeScript) de extremo a extremo: el esquema de datos, las consultas, la lógica de servidor y la interfaz. Prisma genera un cliente tipado a partir de `schema.prisma`, que actúa como fuente única de verdad del modelo de datos.

Este enfoque se aparta del modelo de tres capas separadas (un cliente, un servidor y una base de datos desarrollados como piezas independientes y a menudo en lenguajes distintos) que suele presentarse como referencia. IUCE Reservas adopta en su lugar una arquitectura full-stack integrada en torno a Next.js, en la que el mismo proyecto y lenguaje cubren tanto lo que se ejecuta en el navegador como lo que se ejecuta en el servidor. La ventaja práctica es doble: desaparece la frontera de lenguaje entre frontend y backend (no hay que cambiar de tecnología)

y se elimina buena parte del código repetitivo que normalmente se dedica a comunicar dos aplicaciones distintas.

El hecho de usar un único lenguaje, TypeScript, de extremo a extremo refuerza esa idea. Como señala la documentación oficial de TypeScript, su objetivo es actuar como un comprobador de tipos estático: "The goal of TypeScript is to be a static typechecker for JavaScript programs — in other words, a tool that runs before your code runs (static) and ensures that the types of the program are correct (typechecked)" (Microsoft, s.f.). En la práctica, esto significa que muchos errores (un nombre de campo mal escrito, un dato que se esperaba como número y llega como texto) se detectan al escribir el código y no cuando el usuario ya está utilizando la aplicación. Que ese mismo lenguaje y esos mismos tipos se compartan entre la base de datos, la lógica de servidor y la interfaz aporta una coherencia difícil de lograr cuando cada capa habla un idioma diferente: un cambio en la forma de los datos se propaga y se verifica automáticamente en todos los puntos que lo usan.

La elección de Prisma como ORM, en lugar de escribir consultas SQL a mano, va en la misma dirección. Un ORM es una capa que traduce entre las tablas de la base de datos y los objetos del lenguaje de programación, de modo que el desarrollador trabaja con estructuras tipadas y legibles en vez de con cadenas de SQL sueltas. Por debajo sigue habiendo PostgreSQL y un modelo relacional clásico, de modo que el ORM se apoya en el conocimiento de bases de datos relacionales y lo hace más seguro, en lugar de reemplazarlo. Según la documentación de Prisma (s.f.), el esquema actúa como una única fuente de verdad a partir de la cual se genera un cliente tipado, frente al SQL escrito a mano, que da control pero resulta propenso a errores y carece de seguridad de tipos: el esquema se declara una sola vez en un fichero (`schema.prisma`) y cualquier consulta que no encaje con él se detecta antes de ejecutarse. Esta protección alcanza también a un riesgo clásico, la inyección de SQL; de hecho, OWASP (s.f.) recomienda que "applications built from scratch, or applications requiring low risk tolerance should be built or re-written using parameterized queries, stored procedures, or some kind of Object Relational Mapper (ORM) that builds your queries for you". Se gana así protección frente a errores comunes y se mantiene una única fuente de verdad del modelo de datos, sin renunciar a la potencia de una base de datos relacional madura como PostgreSQL.

Por último, el uso de Docker persigue que el entorno sea reproducible y "todo-en-uno". Según la documentación oficial de Docker (s.f.), "Docker provides the ability to package and run an application in a loosely isolated environment called a container [...]. Containers are lightweight and contain everything needed to run the application, so you don't need to rely on what's installed on the host". En producción, tanto la aplicación como PostgreSQL se ejecutan como contenedores Docker orquestados con `docker-compose` (tras Apache 2 como proxy inverso), de

modo que el despliegue es reproducible y autocontenido. En desarrollo, en cambio, docker-compose levanta únicamente una PostgreSQL idéntica con una sola orden, aislada del resto del sistema y desechable, mientras la aplicación se ejecuta con el servidor de desarrollo de Next.js; así se evita pedir a quien retome el proyecto que instale y configure manualmente una base de datos en su equipo (con el riesgo de que su versión no coincida con la de producción). En ambos casos, el clásico problema de "en mi máquina funciona" se reduce de forma drástica: el entorno de desarrollo se parece al de producción y montarlo es cuestión de minutos. Este conjunto de decisiones (arquitectura full-stack, un solo lenguaje tipado, ORM sobre una base relacional y entorno en contenedor) refleja cómo se construye hoy software web profesional, y su adopción en este proyecto es deliberada: prioriza la coherencia, la seguridad frente a errores y la facilidad de mantenimiento futuro por encima de la familiaridad con esquemas más tradicionales. Como recuerda Sommerville (2016), la mantenibilidad y la fiabilidad son atributos esenciales de cualquier sistema software de calidad, y son precisamente esos atributos los que guían las decisiones de diseño descritas en este apartado.

Sobre la ausencia de diagrama de clases, se ha optado deliberadamente por no incluirlo. IUCE Reservas no sigue un paradigma orientado a objetos clásico, sino una arquitectura funcional y modular propia de Next.js 14 y React: la lógica reside en funciones puras y módulos (carpeta lib/), en componentes de React y en manejadores de ruta (route handlers), y no en una jerarquía de clases instanciables con estado y métodos. La estructura estática del sistema queda, por tanto, mejor representada mediante el modelo de datos o diagrama entidad-relación, que describe las entidades persistentes gestionadas con Prisma, y mediante el diagrama de componentes, que muestra la organización de los módulos de software y sus interfaces. Un diagrama de clases en este contexto resultaría artificial (duplicaría el modelo de datos o forzaría los módulos funcionales a una notación que no siguen) y restaría claridad en lugar de aportarla. En coherencia con el principio de emplear en cada caso la representación más fiel a la naturaleza del sistema, la vista estructural se documenta con el modelo de datos y el diagrama de componentes, y la vista dinámica con los diagramas de flujo, de estados y de secuencia.

Los diagramas que acompañan este apéndice siguen la notación del Lenguaje Unificado de Modelado (UML, Unified Modeling Language), el estándar de modelado de sistemas software más extendido, mantenido por el Object Management Group (OMG) y reconocido como norma internacional ISO/IEC 19505. UML es un lenguaje gráfico para visualizar, especificar, construir y documentar un sistema; no impone una metodología de desarrollo concreta, sino que proporciona una notación común para representar tanto los aspectos estáticos como los dinámicos de un sistema. Como señala Fowler (1999), su valor no reside en dibujar diagramas exhaustivos, sino en emplear selectivamente aquellos que ayudan a comprender y comunicar el diseño.

UML organiza sus diagramas en dos grandes familias: los diagramas estructurales, que describen la organización estática del sistema (entre ellos el diagrama de componentes y, de

forma afin, el modelo de datos), y los diagramas de comportamiento, que describen su dinámica (diagramas de actividad, de estados y de secuencia, entre otros). Este apéndice emplea un subconjunto de ambas familias, escogido según la naturaleza del sistema y descrito en los apartados siguientes.

La tabla siguiente resume los diagramas empleados en este apéndice, clasificados según la vista del sistema que representan:

Tabla 22. *Diagramas empleados en la especificación de diseño.*

Vista	Tipo de diagrama	Figura
Estructural	Modelo de datos (entidad-relación)	Figura 15
Estructural	Diagrama de componentes	Figura 21
Dinámica	Diagrama de flujo (actividad)	Figura 16
Dinámica	Diagrama de estados	Figura 17
Dinámica	Diagrama de secuencia (acceso)	Figura 18
Dinámica	Diagrama de secuencia (aprobación)	Figura 19
Organizativa	Jerarquía de roles y permisos	Figura 20

C.2. Diseño de datos

El modelo relacional se define en schema.prisma sobre PostgreSQL 16. Gira en torno a tres entidades principales (User, Space, Reservation) y un conjunto auxiliar: BlockedSlot, SiteSetting, AuditLog y los modelos estándar de NextAuth (Account, Session, VerificationToken).

C.2.1. Enumeraciones

El modelo emplea dos enumeraciones para acotar los valores admisibles de ciertos campos. La primera fija los roles de usuario y la segunda, los estados por los que puede pasar una reserva. Definirlas a nivel de esquema garantiza la integridad de los datos y evita valores arbitrarios en la base de datos.

- UserRole: USER · ADMIN · SUPER_ADMIN.

- ReservationStatus: PENDING · APPROVED · REJECTED · CANCELLED · EXPIRED.

C.2.2. Entidad User

Representa a las personas que acceden al sistema, siempre identificadas por su correo institucional @usal.es. Además de los datos de identificación, almacena el rol, el estado de la cuenta (incluida la posible suspensión) y marcas temporales de actividad. La tabla siguiente detalla cada campo:

Tabla 23. *Entidad User.*

Campo	Tipo	Req.	Descripción
id	CUID	Sí	Identificador único (clave primaria).
email	String	Sí	Correo @usal.es. Único e indexado.
name	String	Sí	Nombre completo del usuario.
role	Enum	Sí	USER ADMIN SUPER_ADMIN. Por defecto USER.
image	String	No	URL de avatar (opcional).
profileCompleted	Boolean	Sí	Onboarding de perfil completado. Por defecto false.
isBanned	Boolean	Sí	Indica si la cuenta está suspendida. Por defecto false.
banReason	String	No	Motivo de la suspensión.
createdAt	DateTime	Sí	Fecha de creación.
lastLogin	DateTime	No	Fecha del último acceso.
receiveAdminEmails	Boolean	Sí	Solo administradores.

C.2.3. Entidad Space

Modela cada espacio físico reservable del IUCE (aulas, laboratorios o salas), con sus características de aforo, ubicación y equipamiento. Incluye atributos de presentación, como el color en el calendario y la fotografía, y un indicador de baja lógica que permite retirar un espacio sin perder su histórico de reservas. Sus campos son los siguientes:

Tabla 24. *Entidad Space.*

Campo	Tipo	Req.	Descripción
id	CUID	Sí	Identificador único.
name	String	Sí	Nombre del espacio.
code	String	Sí	Código interno único (ej. IUCE-17A).
description	String	No	Descripción del espacio.
capacity	Int	Sí	Aforo máximo.
floor	Int	No	Planta del edificio.
building	String	Sí	Edificio. Por defecto "IUCE".
equipment	Json	Sí	Lista de equipamiento (JSON). Por defecto [].
accessibility	Boolean	Sí	Accesible para movilidad reducida. Por defecto false.
color	String	Sí	Color hex para el calendario. Por defecto #3B7DD8.
imageUrl	String	No	Foto del espacio.
isActive	Boolean	Sí	Permite desactivar (soft delete). Por defecto true.
createdAt / updatedAt	DateTime	Sí	Marcas de tiempo.

C.2.4. Entidad Reservation

Es la entidad central del sistema: cada fila representa una solicitud de uso de un espacio durante una franja horaria concreta. Reúne los datos de la reserva, su estado dentro del flujo de aprobación, la información de revisión administrativa y los campos que dan soporte a las series recurrentes. La tabla recoge el detalle de cada atributo:

Tabla 25. *Entidad Reservation.*

Campo	Tipo	Req.	Descripción
id	CUID	Sí	Identificador único.
title	String	Sí	Título o motivo de la reserva.
description	String	No	Descripción (opcional, máx. 500).
startTime / endTime	DateTime	Sí	Inicio y fin de la franja.
status	Enum	Sí	PENDING APPROVED REJECTED CANCELLED EXPIRED. Por defecto PENDING.
attendees	Int	Sí	Número de asistentes (mín. 1). Por defecto 1.
adminNotes	String	No	Notas del admin al revisar.
reviewedAt	DateTime	No	Fecha de la revisión.
isRecurring	Boolean	Sí	Indica si pertenece a una serie. Por defecto false.
recurrenceRule	String	No	WEEKLY BIWEEKLY MONTHLY.
recurrenceGroupId	String	No	Agrupar las instancias de una serie.
userId	CUID (FK)	Sí	Usuario solicitante.
spaceId	CUID (FK)	Sí	Espacio reservado.
reviewedBy	CUID (FK)	No	Admin que revisó la solicitud.
createdAt / updatedAt	DateTime	Sí	Marcas de tiempo.

C.2.5. Entidades auxiliares

BlockedSlot

Bloqueo administrativo de una franja (mantenimiento, festivos, eventos). Campos: id, startTime, endTime, reason, createdAt, spaceId (FK), createdById (FK). No tiene solicitante ni estado: representa simplemente una franja ocupada.

SiteSetting

Almacén clave-valor para contenido editable. Campos: key (PK), value (Text), updatedAt. Se usa, por ejemplo, para las normas de uso.

AuditLog

Registro inmutable de acciones administrativas. Campos: id, action, userId (FK), targetType, targetId, details (Text/JSON), createdAt.

NextAuth: Account, Session, VerificationToken

Modelos estándar de NextAuth. VerificationToken (identifier, token único, expires) soporta el flujo de magic link.

C.2.6. Modelo de datos: entidades, relaciones e índices.

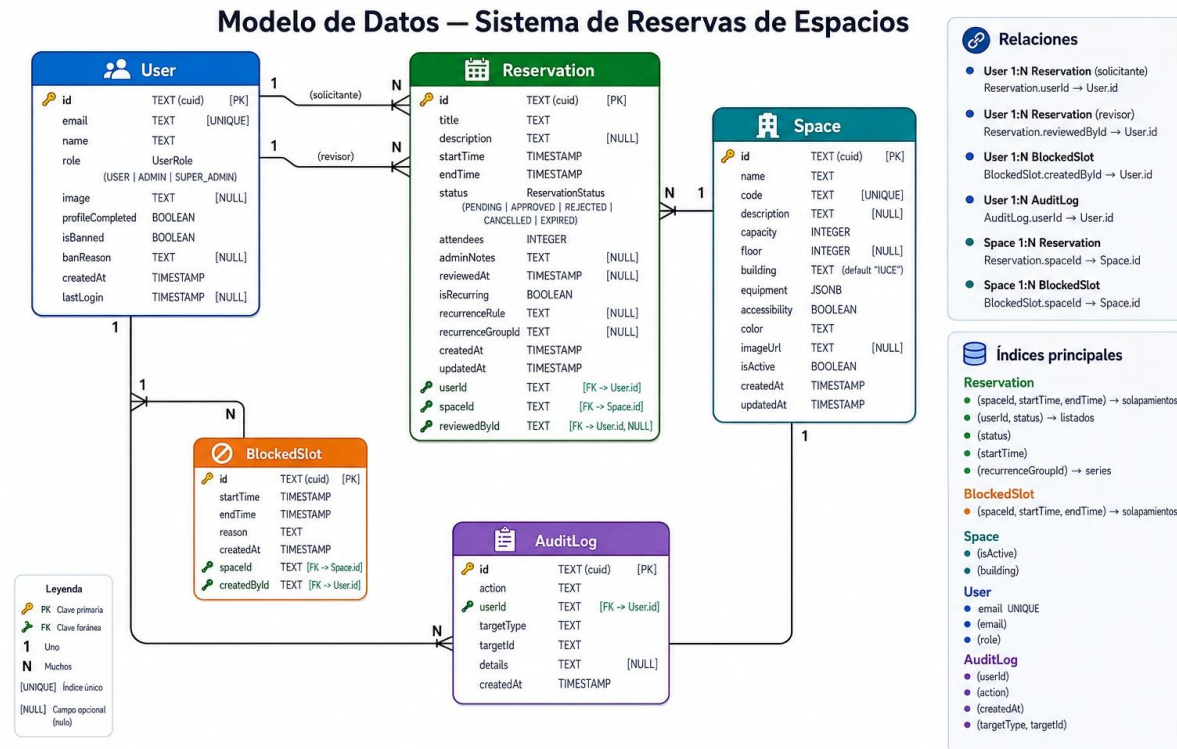


Figura 15. Relaciones e índices del modelo de datos junto con el diagrama de Entidad-Relación de modelado lógico.

C.2.7. Decisiones de diseño de datos

El modelo refleja varias decisiones de diseño tomadas de forma consciente, que conviene explicar porque condicionan cómo se comporta el sistema.

La más relevante tiene que ver con las reservas que se repiten en el tiempo (por ejemplo, un seminario que se imparte todos los martes durante un cuatrimestre). Existían dos maneras de guardarlas. La primera es almacenar una sola “regla de repetición” (una frase del tipo «todos los martes hasta junio») y calcular sobre la marcha, cada vez que alguien consulta el calendario, qué días concretos caen dentro de esa regla. La segunda, que es la elegida, consiste en crear de antemano una fila en la base de datos por cada sesión real: si el seminario tiene quince martes, se guardan quince reservas, una por día. Todas las sesiones de una misma serie quedan enlazadas mediante un identificador común (`recurrenceGroupId`), de modo que el sistema sabe que pertenecen al mismo grupo. Este enfoque tiene dos ventajas prácticas: comprobar si dos reservas se pisan en el mismo espacio y la misma hora se reduce a comparar filas concretas, sin tener que recalcular fechas en cada consulta; y permite cancelar o modificar un día suelto (pongamos, el martes que cae en festivo) sin tocar el resto de la serie.

Una segunda decisión afecta a los bloqueos administrativos, es decir, a los periodos en que un espacio se marca como no disponible (por obras, limpieza o un acto interno) sin que medie ninguna solicitud de un usuario. Aunque un bloqueo “ocupa” un espacio igual que una reserva, no tiene quien lo solicite, ni estado pendiente o aprobado, ni pasa por el circuito de aprobación. Por eso se guarda en su propia tabla (`BlockedSlot`), separada de las reservas. Mezclarlo con ellas obligaría a dejar vacíos muchos campos que un bloqueo no usa y enturbiaría la lógica que controla en qué situación está cada reserva; mantenerlo aparte deja ambos conceptos limpios.

La tercera decisión es el registro de auditoría (`AuditLog`), un histórico de las acciones administrativas relevantes (quién aprobó o rechazó qué y cuándo). Este registro se concibe como inmutable: una vez escrita, una anotación no se modifica ni se borra. Así queda una traza fiable de las decisiones tomadas, útil para rendir cuentas, sin que ese histórico interfiera en el funcionamiento normal del sistema.

Por último, las notificaciones que el usuario ve dentro de la aplicación (avisos del tipo “tu reserva ha sido aprobada”) no se almacenan en ninguna tabla propia. En su lugar, se calculan en el momento de mostrarlas a partir del estado actual de las reservas del usuario. De este modo se evita guardar por duplicado una información que ya está en el modelo y que podría quedar desactualizada respecto a la reserva a la que se refiere.

C.3. Diseño procedimental

Esta sección describe la lógica de negocio del sistema: cómo fluye una reserva desde que se solicita hasta que se resuelve, y qué reglas la gobiernan en cada paso. Se detalla la máquina de estados que rige el ciclo de vida de la reserva, los flujos principales (creación, autenticación y aprobación) y el reparto de responsabilidades entre los distintos roles. El objetivo es dejar trazado el comportamiento del sistema con independencia de los detalles de implementación.

La *Figura 16* ofrece una visión global del ciclo de vida de una reserva, organizada por responsables (usuario, sistema, administrador y tarea programada). El recorrido parte del acceso del usuario mediante magic link y la consulta del catálogo, continúa con la solicitud y su validación automática (límite de peticiones, cuenta activa, aforo, solapamiento y disponibilidad de la franja), y desemboca en la creación de la reserva en estado PENDING. A partir de ahí, el administrador decide su aprobación o rechazo, el usuario puede cancelarla mientras siga vigente y la tarea programada la marca como expirada una vez superada su hora de fin. Las secciones siguientes detallan cada uno de estos subprocesos.

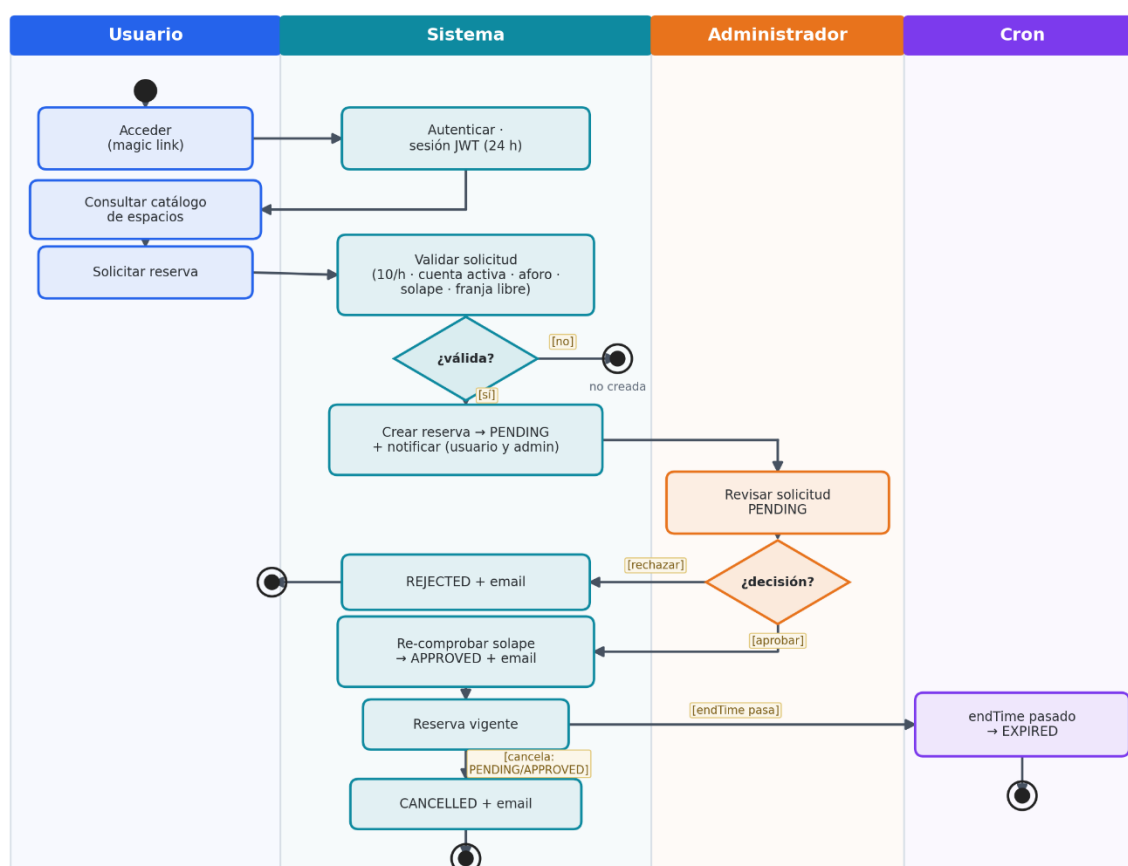


Figura 16. Diagrama de flujo general del ciclo de vida de una reserva.

C.3.1. Máquina de estados de la reserva

Una reserva nace en PENDING. Un administrador la pasa a APPROVED o REJECTED; el usuario (o un admin) puede llevarla a CANCELLED mientras siga vigente (desde PENDING o APPROVED); un administrador puede además anular una reserva ya aprobada (APPROVED → REJECTED), decisión que queda auditada y se notifica por correo; y la tarea programada marca como EXPIRED las reservas APPROVED cuya hora de fin ya ha pasado. REJECTED, CANCELLED y EXPIRED son estados terminales.

La transición PENDING → APPROVED/REJECTED se valida en una función pura (canTransitionTo). La cancelación se gestiona en su propio endpoint y la expiración en el cron. La aprobación, el rechazo y la cancelación disparan el envío del email correspondiente, ejecutado tras confirmar la escritura en base de datos. La expiración, en cambio, se aplica en el cron de forma masiva (updateMany) y no genera notificación.

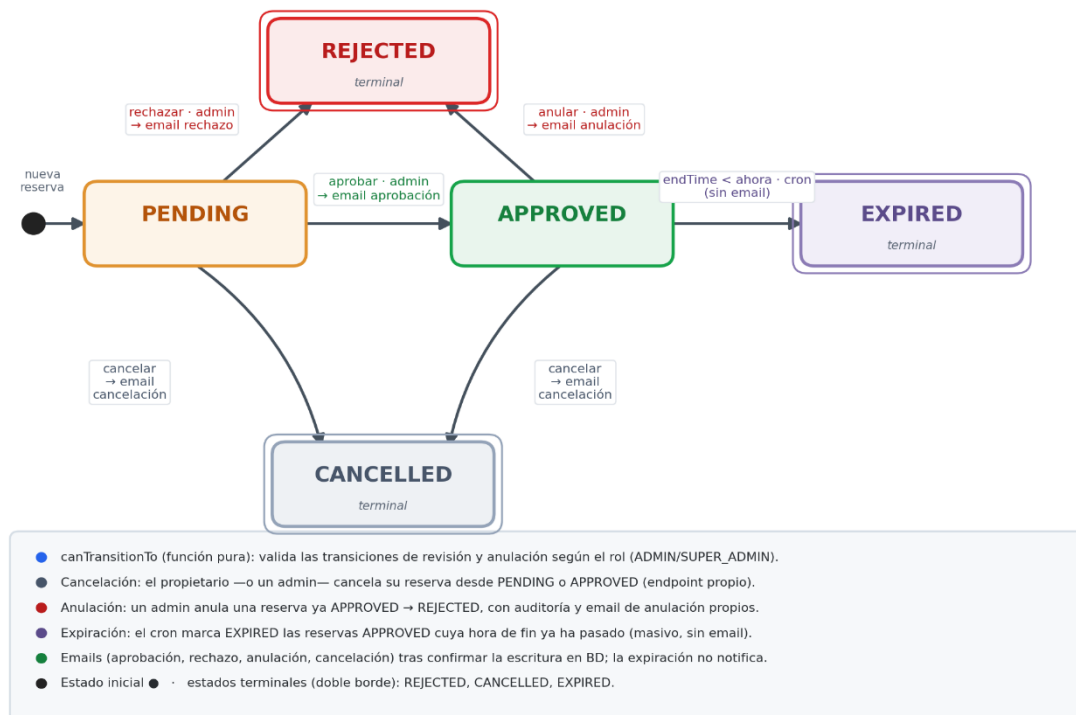


Figura 17. Diagrama de estados de una reserva.

C.3.2. Flujo de creación de una reserva

El endpoint POST /api/reservations aplica, en orden: límite de peticiones (rate limit de 10 reservas por hora y usuario), comprobación de suspensión (un usuario con isBanned recibe 403), validación de los datos con Zod, comprobación de capacidad del espacio, detección de solapamiento con reservas en estado PENDING o APPROVED (409) y comprobación de

bloqueos administrativos (409). Si todo es válido, crea la reserva en PENDING y envía el email de confirmación al usuario y el aviso a los administradores.

Si la solicitud es recurrente (WEEKLY, BIWEEKLY o MONTHLY), el motor expande las fechas (máximo 52 ocurrencias), comprueba cada instancia de forma independiente frente a reservas y bloqueos, crea las válidas compartiendo recurrenceGroupId y devuelve las omitidas en el campo skipped.

C.3.3. Validación de solapamientos y concurrencia

La regla de no solapamiento fijada en el Anexo B se traduce en el diseño en una comprobación concreta sobre intervalos de tiempo. Dos reservas se solapan cuando comparten al menos un instante en el mismo espacio; en lugar de enumerar todas las formas en que dos franjas pueden cruzarse (una contenida en otra, un cruce parcial al inicio o al final, etc.), basta con una única condición: hay conflicto si la reserva existente termina después de que empiece la nueva y, a la vez, empieza antes de que la nueva termine ($\text{fin_existente} > \text{inicio_nuevo}$ y $\text{inicio_existente} < \text{fin_nuevo}$). Esa expresión cubre todos los casos de cruce con una sola comparación. La comprobación solo considera las reservas en estado PENDING o APPROVED, porque las canceladas, rechazadas o expiradas han liberado la franja y no deben bloquearla.

El segundo aspecto, la concurrencia, atiende a qué ocurre cuando dos personas intentan reservar la misma franja casi a la vez. Al crear una reserva, el sistema primero consulta si la franja está libre y, si no hay conflicto, la inserta; como ambas operaciones suceden una tras otra, existe una ventana mínima en la que dos solicitudes simultáneas podrían superar las dos la comprobación antes de que ninguna se haya guardado. Para cubrir ese caso, el sistema vuelve a comprobar el solapamiento en el momento de aprobar, contrastando contra las reservas que ya están APPROVED: aunque dos solicitudes en conflicto lleguen a coexistir como PENDING, nunca llegan a aprobarse las dos. Dado el volumen real de uso del IUCE (del orden de decenas de usuarios concurrentes), la probabilidad de una colisión exacta es muy baja, y esta doble verificación la resuelve sin necesidad de mecanismos de bloqueo más costosos.

C.3.4. Secuencia de autenticación (magic link)

El flujo de autenticación comienza cuando el usuario introduce su correo institucional @usal.es en la ruta /auth/signin. El servidor valida que el dominio sea el esperado, genera un token criptográfico de 32 bytes y lo almacena en VerificationToken con una validez de 24 horas, tras lo cual envía al usuario un enlace de acceso por correo electrónico mediante Resend. Al abrir ese enlace, la ruta /auth/magic-callback invoca a NextAuth a través del proveedor Credentials para verificar el token contra la base de datos. Si el token es válido, se elimina de inmediato para garantizar un único uso, se crea o actualiza el registro del usuario y se establece una sesión

JWT con una duración de 24 horas, redirigiendo finalmente al usuario a su panel principal (dashboard).

Durante el desarrollo puede activarse un proveedor simulado (mock) mediante la variable `NEXT_PUBLIC_USE MOCK_AUTH=true`, que evita el envío real de correos y agiliza las pruebas. El rol del usuario (USER, ADMIN o SUPER_ADMIN) se resuelve en el callback de sesión, mientras que un middleware protege las rutas privadas antes de que la petición alcance su manejador.

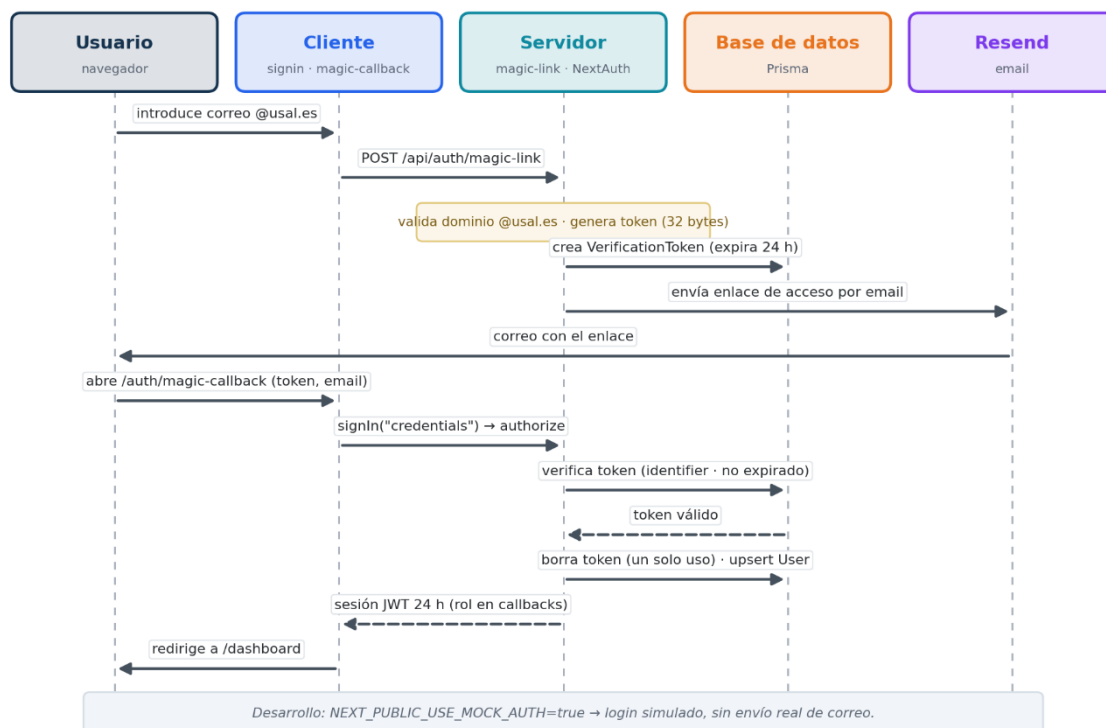


Figura 18. Diagrama de secuencia del acceso mediante magic link.

C.3.5. Secuencia de aprobación administrativa

El administrador revisa las solicitudes PENDING en su panel y decide mediante `PATCH /api/admin/reservations/:id` (body { status, adminNotes }). El sistema valida la transición, re-comprueba el solapamiento al aprobar, registra la decisión en el AuditLog y envía el email de resultado al solicitante.

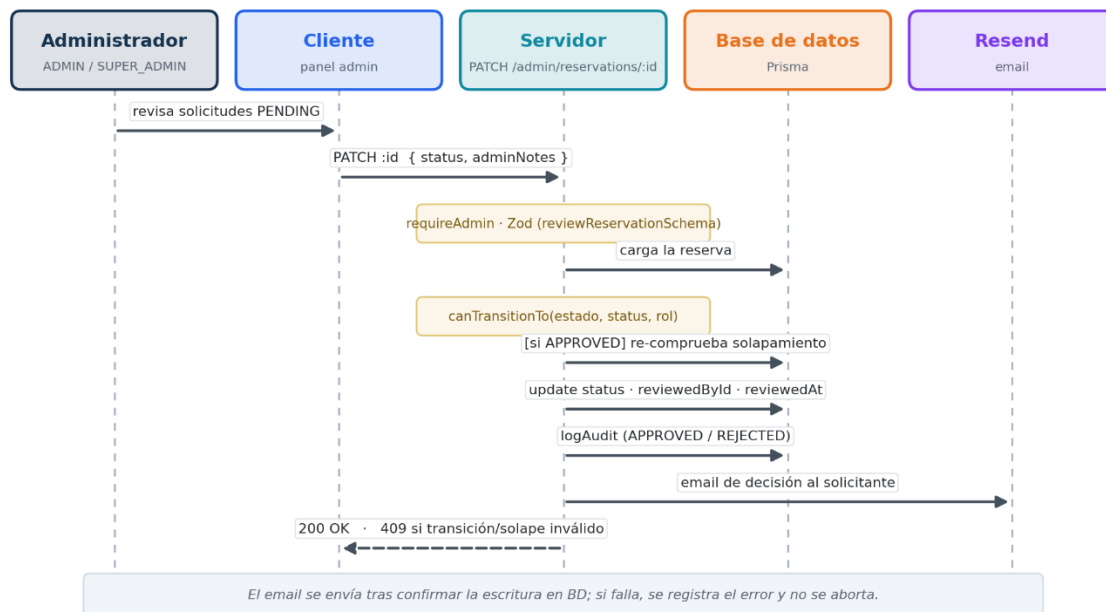


Figura 19. Diagrama de secuencia de la aprobación administrativa.

C.3.6. Procesos por rol

El sistema distingue tres roles. El USER consulta el catálogo, crea y cancela sus reservas y consulta su estado. El ADMIN aprueba/rechaza solicitudes, gestiona espacios y bloqueos, consulta métricas y gestiona usuarios (roles y suspensiones). El SUPER_ADMIN añade una única capacidad exclusiva: modificar el rol de otros SUPER_ADMIN (a quienes, además, ningún administrador puede suspender). La configuración global del sitio, en cambio, puede editarla cualquier administrador. Todas las acciones administrativas quedan registradas en el AuditLog.

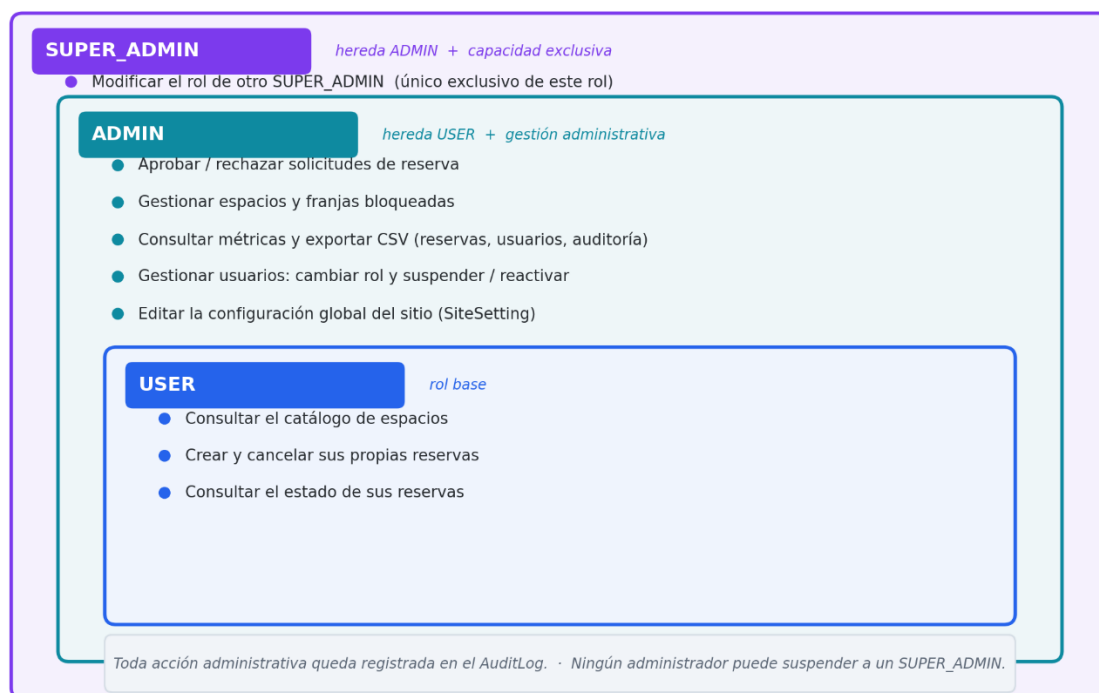


Figura 20. *Jerarquía de roles y herencia de permisos.*

C.4. Diseño arquitectónico

Esta sección presenta la organización técnica del sistema: cómo se estructura el código en capas, cómo se distribuyen las rutas y qué decisiones transversales (presentación, seguridad, rendimiento y despliegue) sostienen la solución. Todo el sistema se construye sobre un único proyecto Next.js, lo que mantiene una base de código homogénea de extremo a extremo. A continuación se desglosan los aspectos arquitectónicos más relevantes tal y como están implementados.

C.4.1. Arquitectura en capas

El sistema se organiza en tres capas con responsabilidades bien diferenciadas: presentación, lógica y API, y datos.

La capa de presentación se apoya en el App Router de Next.js 14: la mayoría de las páginas son Server Components que acceden directamente al cliente de Prisma, mientras que la interactividad (formularios, calendarios y modales) se concentra en Client Components marcados con "use client". La capa de lógica y API la forman las rutas REST bajo /api/ (Next.js API Routes), que encapsulan las mutaciones y la lógica de negocio; las operaciones desde el cliente se realizan mediante fetch contra estas rutas, ya que el sistema no utiliza Server Actions.

Por último, la capa de datos es PostgreSQL 16 a través de Prisma, con el esquema definido en schema.prisma como fuente única de verdad.

La *Figura 21* representa esta arquitectura como un diagrama de componentes, mostrando los módulos de software del sistema, su agrupación (cliente, servidor Next.js y servicios externos) y las interfaces a través de las cuales se comunican.

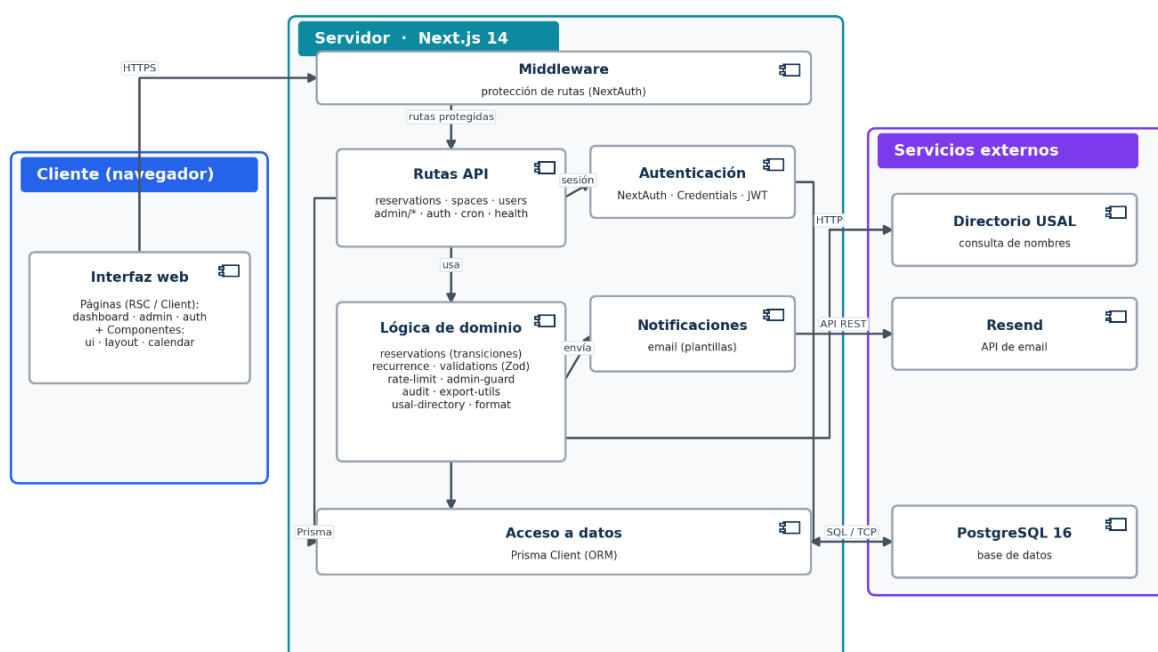


Figura 21. Diagrama de componentes del sistema.

C.4.2. Mapa de rutas

Las rutas de la aplicación se agrupan en tres zonas según el nivel de acceso requerido: pública, de usuario autenticado y de administración. Esta separación se refleja tanto en la estructura del App Router como en el middleware que protege las rutas privadas. La tabla resume las rutas principales de cada zona:

Tabla 26. Mapeado de rutas de la aplicación.

Zona	Rutas
Pública	/ · /auth/signin · /auth/verify-request · /auth/magic-callback
Usuario	/dashboard · /spaces · /spaces/[id] · /spaces/[id]/reserve · /reservations · /reservations/new · /reservations/[id] · /perfil · /normas
Admin	/admin · /admin/reservations · /admin/spaces · /admin/blocked-slots · /admin/users · /admin/normas · /admin/reports

El catálogo de espacios y la API requieren sesión: la página pública de inicio muestra una selección de espacios destacados, pero el catálogo completo y GET /api/spaces exigen autenticación.

C.4.3. Capa de presentación y UX

La interfaz usa Tailwind CSS 3 con componentes propios construidos con class-variance-authority (no se emplea una librería de componentes de terceros). La paleta institucional, definida en la configuración de Tailwind, combina el rojo USAL (#C8102E) y los azules IUCE (#1B3A5C, #3B7DD8) sobre un fondo gris claro, con verde de éxito (#12B76A), ámbar de aviso (#F79009) y rojo de error (#D92D20). El diseño es responsive (mobile-first) e incorpora foco visible (focus-visible) y etiquetas ARIA en los controles interactivos.

C.4.4. Notificaciones

Las notificaciones operan en dos niveles. Los emails transaccionales (Resend) cubren ocho eventos: magic link, confirmación de recepción, aviso a administradores, decisión de aprobación/rechazo, recordatorio 24h, modificación administrativa, cancelación y anulación de reserva aprobada. En desarrollo, sin RESEND_API_KEY válida, el envío se omite y se registra en consola. Las notificaciones in-app (campana) se derivan al vuelo de las reservas (pendientes para el admin, revisadas en los últimos 7 días para el usuario) y se refrescan por polling cada 60 segundos; no se almacenan ni tienen estado de lectura.

C.4.5. Seguridad

La seguridad del sistema se aborda en varios frentes complementarios. La autenticación es passwordless mediante magic link sobre el correo institucional, de modo que el sistema no almacena contraseñas en ningún momento. La autorización se resuelve por rol, combinando un guard de servidor (requireAdmin) con la protección de las rutas privadas en los layouts de

servidor, que redirigen a la pantalla de acceso si no hay sesión, y la verificación de la sesión y el rol en cada handler de la API antes de procesar la petición.

A nivel de uso, un límite de peticiones restringe a diez reservas por hora y usuario, y las cuentas suspendidas reciben una respuesta 403. La entrada se valida en el servidor, con esquemas Zod en las operaciones principales, y las acciones administrativas quedan registradas en el AuditLog. En producción, el sistema se sirve tras Apache con ModSecurity y el conjunto de reglas OWASP CRS, habilitando los métodos PATCH y DELETE para /api/ mediante exclusiones específicas en el VirtualHost.

C.4.6. Rendimiento

El render en Server Components mantiene el JavaScript del cliente reducido (división por ruta). Las consultas críticas de disponibilidad se apoyan en el índice compuesto (spaceId, startTime, endTime). El cuello de botella práctico es el servicio externo de correo, por lo que los envíos se ejecutan de forma asíncrona tras confirmar la escritura en base de datos.

C.4.7. Despliegue

En producción, el sistema se ejecuta en el servidor solis.usal.es del CPD de la USAL, donde Apache 2 actúa como proxy inverso hacia el contenedor de la aplicación Next.js (en localhost:3000); la aplicación y PostgreSQL 16 se ejecutan como contenedores Docker orquestados con docker-compose, y los datos persisten en un volumen Docker. El cifrado en tránsito se resuelve con un certificado SSL de Let's Encrypt renovado automáticamente con certbot, y ModSecurity con el conjunto OWASP CRS permanece activo.

El mantenimiento periódico se apoya en una tarea cron diaria que invoca /api/cron/reminders para enviar los recordatorios y marcar como expiradas las reservas vencidas, protegida mediante un secreto compartido (CRON_SECRET); el endpoint /api/health queda disponible para la monitorización del servicio. En el entorno de desarrollo, en cambio, docker-compose levanta únicamente PostgreSQL y la aplicación se ejecuta con el servidor de desarrollo de Next.js.